

UNIVERSIDAD CIENTÍFICA DEL SUR

S.A.P.S

Sistema Administrador de Proyectos Son

Proyecto integrador de aplicaciones web con Spring Boot

Curso	Lenguaje de Programación 2
Estudiante	Orihuela Jara, Edelson Anghuelo
Docente	Julio Alfredo Córdova Forero
Semestre	2026 - 1
Lugar	Lima, Perú
Tipo de documento	Informe final del proyecto

Lima, Perú

Índice

N.º	Sección
Resumen ejecutivo	Síntesis del propósito, alcance y resultados del proyecto.
1	Introducción
2	Planteamiento del problema
3	Objetivos del proyecto
4	Justificación
5	Alcance del sistema
6	Marco tecnológico
7	Arquitectura del sistema
8	Análisis y diseño del sistema
9	Modelo de base de datos
10	Diccionario de datos
11	Descripción de módulos implementados
12	Implementación técnica
13	Interfaces del sistema
14	Diagramas UML
15	Pruebas del sistema
16	Resultados obtenidos
17	Alineación con la rúbrica de evaluación
18	Limitaciones y mejoras futuras
19	Conclusiones
20	Recomendaciones
21	Referencias

Resumen ejecutivo

El presente informe documenta el desarrollo de S.A.P.S, Sistema Administrador de Proyectos Son, una aplicación web académica orientada a centralizar la administración de proyectos integradores universitarios. El sistema permite organizar información relacionada con cursos, docentes, estudiantes, grupos, proyectos, entregables, observaciones, rúbricas, criterios de evaluación, calificaciones y reportes, aplicando una arquitectura MVC con Spring Boot, Thymeleaf, Spring Data JPA y MySQL.

La propuesta responde a una problemática frecuente en contextos académicos: la dispersión de información y la falta de trazabilidad durante el seguimiento y la evaluación de proyectos. A partir de ello, se diseñó una solución web monolítica, organizada por capas, que separa la presentación, la lógica de negocio, el acceso a datos y el modelo de dominio. Esta separación permite evidenciar los aprendizajes del curso Lenguaje de Programación 2 y facilita la sustentación técnica del sistema.

Como resultado, se obtuvo una aplicación funcional con autenticación, módulos CRUD, relaciones JPA, vistas Thymeleaf, dashboard, reportes, datos iniciales, documentación complementaria y diagramas UML. El proyecto no solo demuestra la construcción de pantallas web, sino también la integración de componentes técnicos necesarios para administrar un proceso académico completo.

1. Introducción

En este informe presento el desarrollo de S.A.P.S, Sistema Administrador de Proyectos Son, una aplicación web académica construida para organizar la gestión de proyectos integradores universitarios. El sistema surge a partir de una necesidad frecuente en el trabajo académico: contar con una herramienta que permita registrar proyectos, controlar entregables, centralizar observaciones, administrar rúbricas y consultar resultados desde una misma plataforma.

El proyecto fue desarrollado en el contexto del curso Lenguaje de Programación 2 como una aplicación web basada en Java 17, Spring Boot 3, Spring MVC, Thymeleaf, Spring Data JPA y MySQL. La propuesta aplica el patrón Modelo-Vista-Controlador y una arquitectura por capas para separar la presentación, la lógica de negocio, el acceso a datos y el modelo de dominio. Esta organización facilita la comprensión del flujo de ejecución y permite que cada módulo pueda mantenerse y ampliarse de manera ordenada.

Desde el punto de vista académico, S.A.P.S evidencia el uso integrado de tecnologías backend, vistas server-side, persistencia relacional, validaciones, seguridad básica y documentación técnica. El valor del proyecto no se limita a construir pantallas CRUD, sino a representar el ciclo de vida de un proyecto integrador: desde la organización de cursos y grupos hasta el seguimiento de entregables, observaciones y calificaciones.

2. Planteamiento del problema

En los proyectos integradores universitarios se suele trabajar con diferentes fuentes de información: hojas de cálculo, mensajes, archivos sueltos, observaciones verbales y documentos entregados por separado. Esta forma de trabajo puede funcionar en grupos pequeños, pero se vuelve difícil de controlar cuando intervienen varios cursos, docentes, estudiantes, grupos y criterios de evaluación.

El problema central identificado es la falta de trazabilidad en la administración de proyectos académicos integradores. Cuando la información se encuentra dispersa, resulta complicado conocer el estado real de cada proyecto, revisar los entregables pendientes, registrar observaciones de manera ordenada y sustentar una calificación con base en criterios definidos.

Entre las principales causas se encuentran la ausencia de una plataforma centralizada, el uso de registros manuales, la duplicidad de información y la falta de una estructura única para relacionar proyectos, grupos, docentes, entregables, rúbricas y calificaciones. Como consecuencia, se incrementa el riesgo de pérdida de información, se dificulta el seguimiento de avances y se reduce la claridad del proceso de evaluación.

La formulación del problema se expresa de la siguiente manera: ¿cómo desarrollar una aplicación web académica que permita administrar de forma centralizada los proyectos integradores universitarios, sus entregables, observaciones, rúbricas, calificaciones y reportes, aplicando Spring Boot, Thymeleaf, Spring Data JPA y arquitectura MVC?

3. Objetivos del proyecto

3.1. Objetivo general

Desarrollar una aplicación web MVC con Spring Boot que permita administrar proyectos académicos integradores, gestionando cursos, docentes, estudiantes, grupos, entregables, observaciones, rúbricas, calificaciones y reportes mediante una base de datos MySQL.

3.2. Objetivos específicos

- Gestionar información académica base relacionada con cursos, docentes, estudiantes y grupos de trabajo.
- Administrar proyectos integradores mediante formularios, listados, estados y vistas de detalle.
- Registrar entregables asociados a proyectos, incluyendo fechas, tipo, estado y URL de evidencia.
- Registrar observaciones docentes vinculadas a proyectos y, opcionalmente, a entregables específicos.
- Definir rúbricas y criterios de evaluación con puntajes y pesos porcentuales.
- Registrar calificaciones y calcular la nota final cuando existan detalles de evaluación por criterio.
- Visualizar indicadores generales mediante un dashboard y consultar reportes del sistema.
- Aplicar arquitectura MVC y arquitectura por capas para separar responsabilidades técnicas.
- Integrar Spring Data JPA con MySQL para persistir entidades y relaciones del dominio.

4. Justificación

4.1. Justificación académica

El proyecto permite demostrar los contenidos principales del curso Lenguaje de Programación 2 porque integra desarrollo backend, construcción de vistas, manejo de formularios, persistencia de datos, validaciones y seguridad. Además, me permitió aplicar un flujo completo de desarrollo web con Spring Boot, desde la creación de entidades JPA hasta la presentación de información en Thymeleaf.

4.2. Justificación tecnológica

Spring Boot fue elegido porque reduce la configuración inicial del proyecto y permite levantar una aplicación web ejecutable. Thymeleaf se utilizó para construir vistas dinámicas integradas con el modelo MVC. Spring Data JPA permitió implementar repositorios con menos código repetitivo, mientras que MySQL se ajusta al modelo relacional del sistema. Bootstrap 5 ayudó a mantener una interfaz ordenada y responsiva.

4.3. Justificación funcional

S.A.P.S mejora una gestión manual porque centraliza información y permite relacionar entidades que normalmente se controlan por separado. El sistema facilita consultar proyectos, revisar entregables pendientes, registrar observaciones, mantener rúbricas y observar datos generales desde el dashboard. Esto aporta mayor organización y una base más clara para sustentar el avance académico.

5. Alcance del sistema

El alcance funcional incluye autenticación, administración de usuarios, cursos, docentes, estudiantes, grupos, proyectos, entregables, observaciones, rúbricas, criterios, calificaciones, dashboard y reportes. El alcance técnico comprende una aplicación web monolítica con Spring Boot, vistas Thymeleaf, repositorios JPA y base de datos MySQL.

Tabla 1. Alcance funcional del sistema.

Módulo	Funcionalidades implementadas
Autenticación	Login personalizado, logout, usuarios iniciales y roles ADMIN, DOCENTE y ESTUDIANTE.

Módulo	Funcionalidades implementadas
Usuarios	Listado, creación, edición y eliminación de cuentas. La ruta /usuarios/** está restringida al rol ADMIN.
Cursos	Mantenimiento de código, nombre, ciclo, créditos, descripción y estado.
Docentes	Registro de datos personales, correo, especialidad, teléfono y estado.
Estudiantes	Registro de código universitario, datos personales, carrera, ciclo, estado y grupo.
Grupos	Asignación de curso y docente asesor, listado, formulario y vista de detalle con integrantes.
Proyectos	Registro de información del proyecto, curso, grupo, docente asesor, fechas y estado.
Entregables	Control de nombre, tipo, descripción, fechas, URL de archivo y estado.
Observaciones	Registro de comentario, nivel, fecha, estado, proyecto, entregable y docente.
Rúbricas y criterios	Mantenimiento de rúbricas por curso y criterios con puntaje, peso y orden.
Calificaciones	Registro de proyecto, rúbrica, docente, nota final, comentario, estado y vista de detalle.
Dashboard y reportes	Indicadores generales y reporte con cursos, grupos, proyectos, entregables, observaciones y calificaciones.

Las restricciones principales son el uso de una base de datos MySQL local, la ejecución en ambiente de desarrollo mediante XAMPP y el uso de credenciales iniciales de prueba. Como limitaciones reales, el sistema no exporta reportes a PDF o Excel, no implementa carga física de archivos y no posee notificaciones por correo. Estas funciones se consideran mejoras futuras.

6. Marco tecnológico

Tabla 2. Tecnologías utilizadas en el proyecto.

Tecnología	Uso en el proyecto
Java 17	Lenguaje base del backend y versión definida para la ejecución del proyecto.
Spring Boot 3.3.5	Arranque de la aplicación, autoconfiguración, servidor embebido y organización general.
Spring MVC	Controladores, rutas HTTP, formularios, modelos y retorno de vistas.
Thymeleaf	Plantillas HTML dinámicas, formularios, listados, fragmentos y expresiones th:*
Spring Data JPA	Repositorios JpaRepository, consultas derivadas y persistencia de entidades.
Spring Security	Login personalizado, logout, codificación de contraseñas y protección de rutas.
MySQL	Base de datos relacional saps_db para almacenar entidades y relaciones.
XAMPP	Entorno local usado para levantar el servicio MySQL durante pruebas.
MySQL Workbench	Herramienta de administración para crear la base y revisar tablas.
Maven	Gestión de dependencias, ciclo de compilación y empaquetado.
Bootstrap 5	Diseño visual, tablas, botones, formularios, badges y componentes responsivos.
HTML5 y CSS3	Estructura y estilos de las vistas del sistema.
Git y GitHub	Control de versiones y repositorio remoto del proyecto.

7. Arquitectura del sistema

7.1. Arquitectura MVC

El sistema aplica el patrón Modelo-Vista-Controlador. El usuario interactúa con vistas Thymeleaf; estas envían solicitudes a controladores Spring MVC; los controladores preparan el modelo y delegan la lógica a

servicios; los servicios usan repositorios JPA para consultar o persistir información; finalmente, la respuesta se devuelve a una plantilla Thymeleaf.

Flujo aplicado: Usuario → Vista Thymeleaf → Controlador → Servicio → Repositorio → Base de datos → Respuesta.

7.2. Arquitectura por capas

- Capa de presentación: plantillas HTML ubicadas en `src/main/resources/templates`.
- Capa controladora: clases del paquete `com.son.saps.controller`.
- Capa de servicio: interfaces en `service` e implementaciones en `service.impl`.
- Capa de persistencia: repositorios que extienden `JpaRepository`.
- Capa de datos: entidades JPA y base de datos MySQL `saps_db`.

7.3. Estructura de paquetes

Tabla 3. Estructura de paquetes del sistema.

Paquete	Descripción dentro del proyecto
<code>com.son.saps</code>	Clase principal <code>SapsApplication</code> .
<code>config</code>	Configuración de seguridad, datos iniciales, MVC y mensajes de arranque.
<code>controller</code>	Controladores MVC para autenticación, CRUDs, dashboard y reportes.
<code>controller.base</code>	Controlador genérico reutilizado por los módulos CRUD.
<code>model</code>	Entidades JPA del dominio académico.
<code>model.enums</code>	Enumeraciones de estados, roles, niveles y tipos de entregable.
<code>repository</code>	Interfaces <code>JpaRepository</code> para acceso a datos.
<code>service</code>	Contratos de servicios y CRUD genérico.
<code>service.impl</code>	Implementaciones de servicios de negocio.

En la versión documentada no se consideran paquetes `dto` ni `util` como parte implementada, por lo que no se presentan dentro de la estructura funcional del informe.

8. Análisis y diseño del sistema

8.1. Requerimientos funcionales

Tabla 4. Matriz de requerimientos funcionales.

Código	Requerimiento	Descripción	Módulo
RF01	Gestionar usuarios	Administrar cuentas, roles, estado y datos de contacto.	Usuarios
RF02	Gestionar cursos	Registrar cursos con código, nombre, ciclo, créditos y estado.	Cursos
RF03	Gestionar docentes	Registrar docentes asesores y sus datos académicos.	Docentes
RF04	Gestionar estudiantes	Registrar estudiantes y asociarlos a grupos.	Estudiantes
RF05	Gestionar grupos	Crear grupos, asociarlos a curso, docente y estudiantes.	Grupos
RF06	Gestionar proyectos	Registrar proyectos integradores, objetivos, fechas y estado.	Proyectos
RF07	Gestionar entregables	Controlar entregables por proyecto con fechas, tipo, URL y estado.	Entregables
RF08	Registrar observaciones	Guardar comentarios docentes por proyecto y entregable.	Observaciones
RF09	Gestionar rúbricas	Definir rúbricas asociadas a cursos.	Rúbricas

Código	Requerimiento	Descripción	Módulo
RF10	Gestionar criterios	Registrar criterios con puntaje máximo, peso y orden.	Criterios
RF11	Registrar calificaciones	Registrar nota final, comentario, estado y datos de evaluación.	Calificaciones
RF12	Visualizar dashboard	Mostrar indicadores y listas resumidas del estado académico.	Dashboard
RF13	Consultar reportes	Reunir información general de cursos, grupos, proyectos y evaluaciones.	Reportes

8.2. Requerimientos no funcionales

Tabla 5. Requerimientos no funcionales.

Categoría	Requerimiento
Seguridad	Autenticación con Spring Security, logout y restricción de usuarios al rol ADMIN.
Usabilidad	Vistas con Bootstrap, navegación lateral, formularios y mensajes de confirmación.
Mantenibilidad	Separación por capas, servicios genéricos y controladores reutilizables.
Compatibilidad	Ejecución local con Java 17, Maven, MySQL y navegador web.
Rendimiento	Consultas simples mediante repositorios JPA para un entorno académico local.
Escalabilidad	Estructura preparada para ampliar módulos y reglas de seguridad por rol.
Integridad de datos	Validaciones con Jakarta Validation y relaciones JPA entre entidades.

8.3. Actores del sistema

Actor	Responsabilidad
Administrador	Gestiona usuarios y puede acceder al mantenimiento general del sistema.
Docente	Usuario previsto para revisar proyectos, registrar observaciones y evaluar mediante rúbricas.
Estudiante	Usuario previsto para consultar información académica relacionada con su proyecto y entregables.

Técnicamente, los roles ADMIN, DOCENTE y ESTUDIANTE existen como datos y autoridades de Spring Security. En la configuración actual, la restricción específica implementada está sobre /usuarios/** para ADMIN; las demás rutas requieren autenticación general.

8.4. Casos de uso principales

Tabla 6. Casos de uso principales.

Caso de uso	Actor principal	Descripción
Iniciar sesión	Administrador, docente o estudiante	Ingresar con credenciales registradas y acceder al dashboard.
Mantener catálogos académicos	Administrador	Gestionar cursos, docentes, estudiantes y grupos.
Registrar proyecto	Usuario autenticado	Crear o editar información del proyecto integrador.
Registrar entregable	Usuario autenticado	Guardar datos de un entregable asociado a un proyecto.
Registrar observación	Usuario autenticado	Agregar comentario, nivel y estado de revisión.

Caso de uso	Actor principal	Descripción
Gestionar rúbrica	Usuario autenticado	Crear rúbricas y criterios de evaluación.
Registrar calificación	Usuario autenticado	Guardar evaluación del proyecto y consultar detalle.
Consultar reportes	Usuario autenticado	Visualizar información consolidada de módulos principales.

9. Modelo de base de datos

El modelo de base de datos se construye mediante entidades JPA y se refleja en MySQL. Cada entidad posee una clave primaria Long con generación IDENTITY. Las relaciones se modelan con @ManyToOne, @OneToMany y @OneToOne, de acuerdo con la necesidad de vincular cursos, grupos, proyectos, entregables, observaciones, rúbricas y calificaciones.

Tabla 7. Entidades principales del modelo de datos.

Entidad	Descripción
Usuario	Cuenta de acceso al sistema.
Rol	Autoridad asignada al usuario.
Curso	Curso académico donde se desarrollan proyectos.
Docente	Docente asesor o evaluador.
Estudiante	Alumno integrante de un grupo.
Grupo	Equipo de trabajo asociado a un curso y docente.
Proyecto	Proyecto integrador con objetivos, fechas y estado.
Entregable	Evidencia o avance asociado a un proyecto.
Observación	Comentario docente sobre proyecto o entregable.
Rúbrica	Instrumento de evaluación por curso.
CriterioEvaluacion	Criterio ponderado de una rúbrica.
Calificación	Evaluación general de un proyecto.
DetalleCalificacion	Puntaje por criterio dentro de una calificación.

Tabla 8. Relaciones principales del modelo de datos.

Relación	Cardinalidad	Implementación
Curso - Grupo	1 a muchos	Curso tiene grupos; Grupo referencia curso.
Curso - Proyecto	1 a muchos	Proyecto pertenece a un curso.
Curso - Rúbrica	1 a muchos	Rúbrica pertenece a un curso.
Docente - Grupo	1 a muchos	Grupo referencia docenteAsesor.
Docente - Proyecto	1 a muchos	Proyecto referencia docenteAsesor.
Grupo - Estudiante	1 a muchos	Estudiante referencia grupo.
Grupo - Proyecto	1 a 1	Proyecto referencia grupo con columna única.
Proyecto - Entregable	1 a muchos	Entregable referencia proyecto.
Proyecto - Observación	1 a muchos	Observación referencia proyecto.
Entregable - Observación	1 a muchos opcional	Observación puede referenciar entregable.
Rúbrica - CriterioEvaluacion	1 a muchos	Criterio referencia rúbrica.
Proyecto - Calificación	1 a muchos	Calificación referencia proyecto.
Calificación - DetalleCalificacion	1 a muchos	Detalle referencia calificación.

10. Diccionario de datos

El diccionario de datos resume las entidades principales, sus tablas, campos más relevantes y relaciones. Esta información facilita comprender cómo se estructura la persistencia del sistema y cómo se conectan los módulos funcionales.

Tabla 9A. Diccionario de datos: entidades base.

Entidad	Tabla	Campos principales	Relaciones
Usuario	usuarios	id; username; password; nombreCompleto; correo; rol; estado; fechaCreacion	ManyToOne con Rol.
Rol	roles	id; nombre; descripcion	Usado por Usuario.
Curso	cursos	id; codigo; nombre; ciclo; credits; descripcion; estado	OneToMany con Grupo, Proyecto y Rubrica.
Docente	docentes	id; nombres; apellidos; correo; especialidad; telefono; estado	OneToMany con Grupo y Proyecto.
Estudiante	estudiantes	id; codigoUniversitario; nombres; apellidos; correo; carrera; ciclo; estado; grupo	ManyToOne con Grupo.
Grupo	grupos	id; nombre; codigoGrupo; curso; docenteAsesor; estado; fechaCreacion	ManyToOne con Curso y Docente; OneToMany con Estudiante; OneToOne con Proyecto.
Proyecto	proyectos	id; titulo; descripcion; problemaIdentificado; objetivoGeneral; objetivosEspecificos; lineaTematica; curso; grupo; docenteAsesor; fechas; estado	ManyToOne con Curso y Docente; OneToOne con Grupo; OneToMany con Entregable, Observacion y Calificacion.

Tabla 9B. Diccionario de datos: seguimiento y evaluación.

Entidad	Tabla	Campos principales	Relaciones
Entregable	entregables	id; proyecto; nombre; tipo; descripcion; fechaLimite; fechaEntrega; urlArchivo; estado	ManyToOne con Proyecto; OneToMany con Observacion.
Observacion	observaciones	id; proyecto; entregable; docente; comentario; nivel; fechaObservacion; estado	ManyToOne con Proyecto, Entregable y Docente.
Rubrica	rubricas	id; nombre; curso; descripcion; puntajeMaximo; estado	ManyToOne con Curso; OneToMany con CriterioEvaluacion.
CriterioEvaluacion	criterios_evaluacion	id; rubrica; nombre; descripcion; puntajeMaximo; pesoPorcentual; orden	ManyToOne con Rubrica.
Calificacion	calificaciones	id; proyecto; rubrica; docenteEvaluador; fechaEvaluacion; notaFinal; comentarioGeneral; estado	ManyToOne con Proyecto, Rubrica y Docente; OneToMany con DetalleCalificacion.
DetalleCalificacion	detalle_calificaciones	id; calificacion; criterio; puntajeObtenido; comentario	ManyToOne con Calificacion y CriterioEvaluacion.

11. Descripción de módulos implementados

11.1. Módulo de autenticación y roles

Implementa una página de login personalizada, procesamiento de credenciales con Spring Security y cierre de sesión. El sistema carga usuarios iniciales y roles ADMIN, DOCENTE y ESTUDIANTE. Las contraseñas generadas por DataInitializer se almacenan mediante el PasswordEncoder delegante de Spring Security.

11.2. Módulo de usuarios

Permite listar, crear, editar y eliminar usuarios. Incluye datos de acceso, nombre completo, correo, rol y estado. Esta ruta está protegida para el rol ADMIN.

11.3. Módulo de cursos

Permite mantener cursos con código único, nombre, ciclo, créditos, descripción y estado. Es la base para relacionar grupos, proyectos y rúbricas.

11.4. Módulo de docentes

Administra docentes con nombres, apellidos, correo, especialidad, teléfono y estado. Los docentes pueden quedar asociados a grupos y proyectos como asesores.

11.5. Módulo de estudiantes

Gestiona estudiantes mediante código universitario, datos personales, correo, carrera, ciclo, estado y grupo asignado.

11.6. Módulo de grupos

Permite crear grupos de trabajo, asociarlos a un curso y a un docente asesor. Cuenta con vista de detalle para revisar integrantes.

11.7. Módulo de proyectos

Administra proyectos integradores con título, descripción, problema identificado, objetivos, línea temática, curso, grupo, docente asesor, fechas y estado.

11.8. Módulo de entregables

Registra evidencias o avances por proyecto. El sistema guarda tipo, fechas, estado y una URL de archivo. En la versión actual no realiza carga física de archivos.

11.9. Módulo de observaciones

Permite registrar comentarios docentes asociados a un proyecto y, si corresponde, a un entregable. También registra nivel, fecha y estado de la observación.

11.10. Módulo de rúbricas y criterios

Mantiene rúbricas vinculadas a cursos y criterios con puntaje máximo, peso porcentual y orden. Esto estructura la evaluación académica del proyecto.

11.11. Módulo de calificaciones

Registra evaluación por proyecto, rúbrica, docente evaluador, fecha, nota final, comentario y estado. El servicio contiene una fórmula para calcular nota final ponderada cuando existen detalles de calificación asociados a criterios.

11.12. Dashboard y reportes

El dashboard muestra indicadores generales, proyectos recientes, próximos entregables, observaciones pendientes y mejores proyectos según nota. El reporte general consolida cursos, grupos, proyectos, entregables, observaciones y calificaciones.

12. Implementación técnica

La implementación utiliza controladores específicos para cada módulo y un BaseCrudController que reduce repetición en listados, formularios, guardado, edición y eliminación. Los servicios siguen una estructura similar mediante GenericCrudServiceImpl, que centraliza operaciones comunes sobre JpaRepository.

Tabla 10. Componentes técnicos principales.

Componente	Ejemplos	Responsabilidad
Controladores	CursoController, ProyectoController, RubricaController, CalificacionController, ReporteController	Recibir solicitudes HTTP, preparar modelos y devolver vistas Thymeleaf.
Servicios	CursoServiceImpl, ProyectoServiceImpl, DashboardServiceImpl, CalificacionServiceImpl	Aplicar lógica de negocio y coordinar acceso a repositorios.
Repositorios	CursoRepository, ProyectoRepository, EntregableRepository, CalificacionRepository	Persistir y consultar entidades con Spring Data JPA.
Entidades	Curso, Grupo, Proyecto, Entregable, Observacion, Rubrica, Calificacion	Representar el modelo de dominio y sus relaciones.
Configuración	SecurityConfig, DataInitializer, WebConfig, StartupLogger	Seguridad, datos iniciales, MVC y mensajes de arranque.

La conexión se define en application.properties mediante jdbc:mysql://localhost:3306/saps_db, usuario root y serverTimezone=America/Lima. Hibernate está configurado con spring.jpa.hibernate.ddl-auto=update, por lo que las tablas se crean o actualizan al iniciar la aplicación.

Las validaciones se implementan con anotaciones de Jakarta Validation como @NotBlank, @NotNull, @Email, @Min, @Max, @DecimalMin y @DecimalMax. Estas validaciones se activan en el controlador base mediante @Valid y BindingResult.

En términos de seguridad, Spring Security gestiona la autenticación, el cierre de sesión y la protección de rutas. Para la sustentación, es importante explicar que el proyecto implementa una restricción específica sobre el mantenimiento de usuarios y que la autorización por rol puede extenderse a otros módulos en futuras versiones.

13. Interfaces del sistema

Esta sección reserva espacios para insertar capturas reales del sistema ejecutado en <http://localhost:8080>. Cada captura debe tomarse desde el navegador con XAMPP y MySQL activos.

Figura 1. Pantalla de inicio de sesión del sistema S.A.P.S.

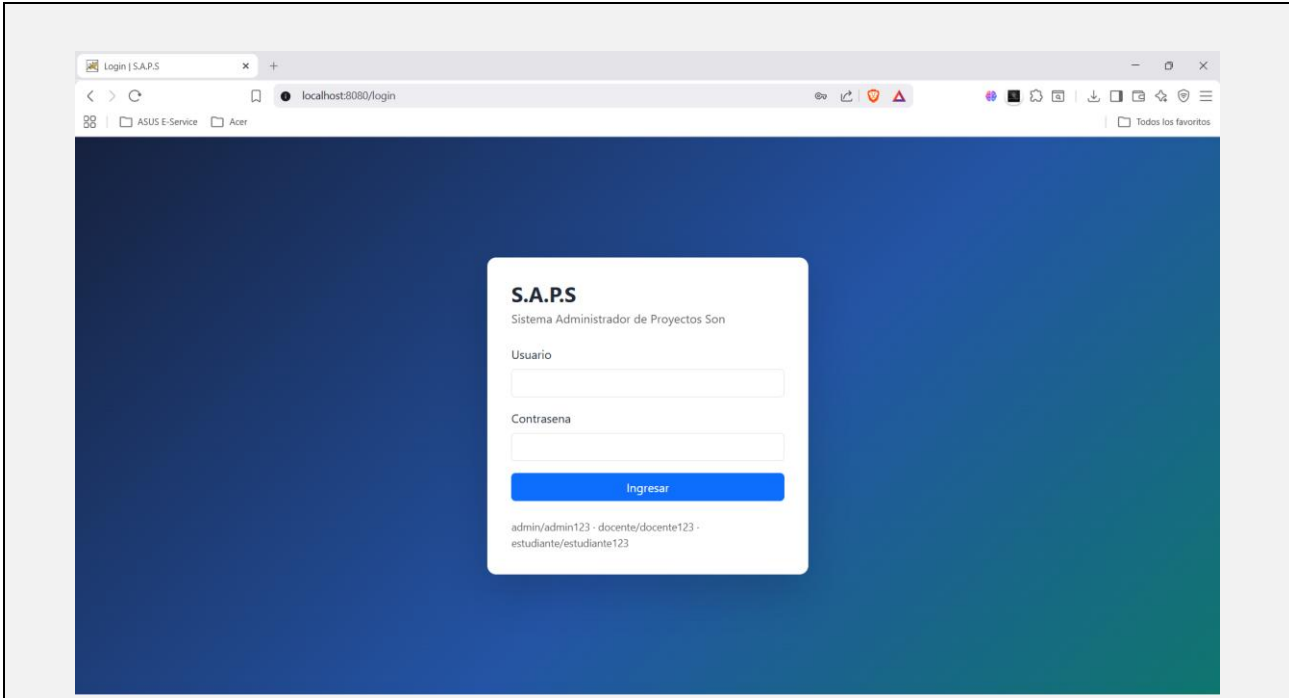


Figura 2. Dashboard principal.

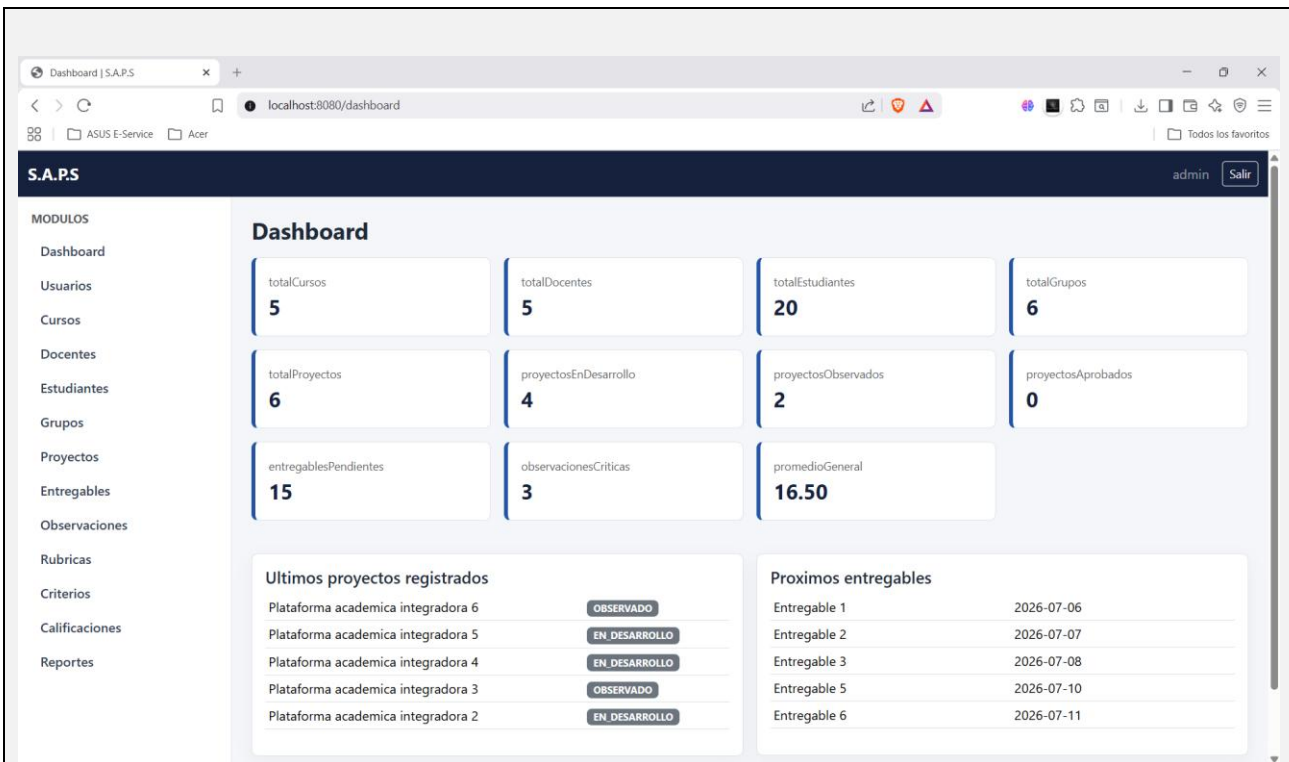


Figura 3. Listado de proyectos.

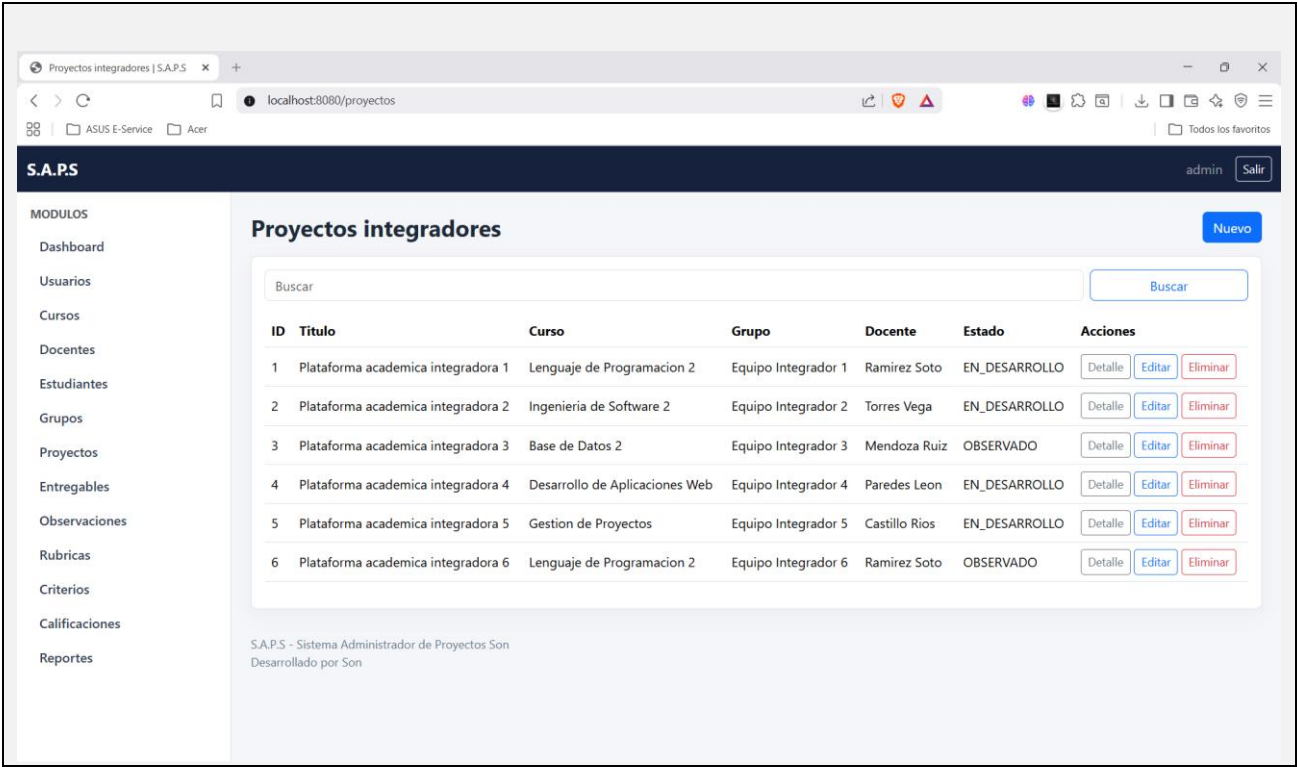


Figura 4. Formulario de proyecto.

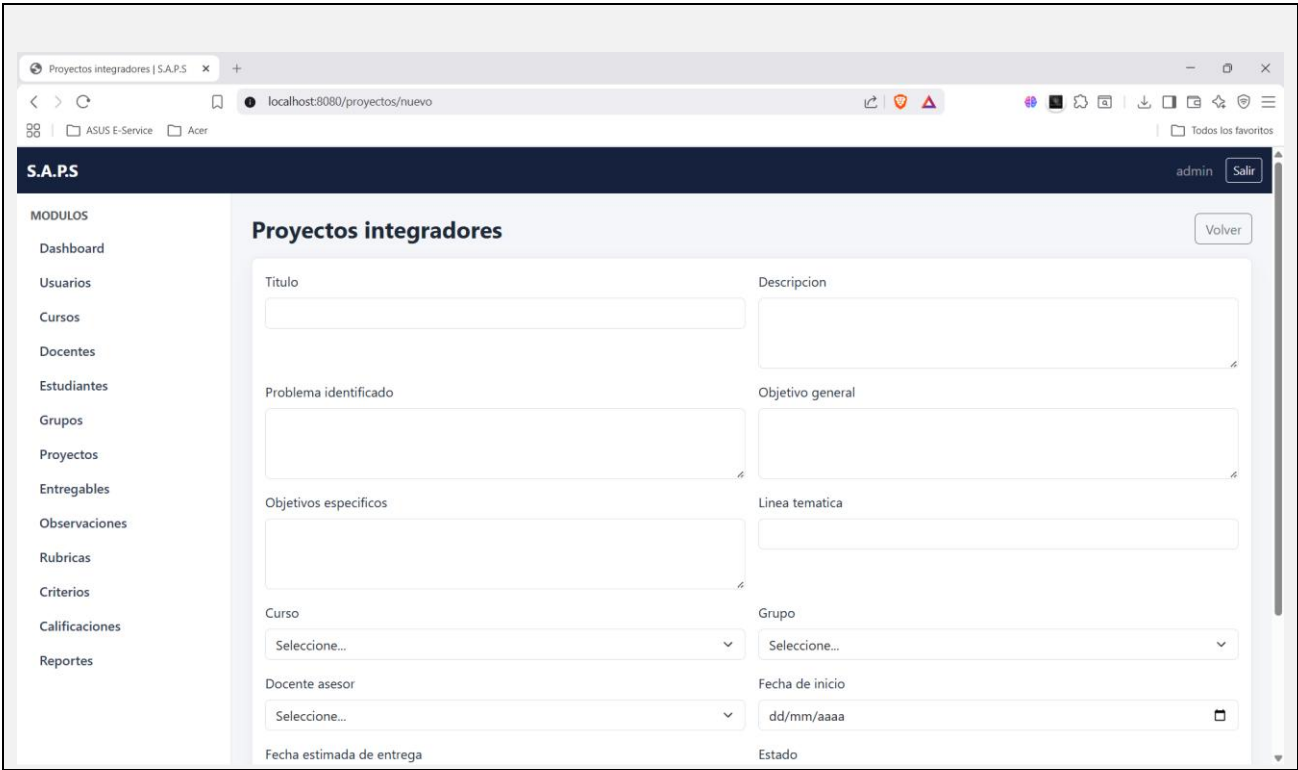


Figura 5. Gestión de entregables.

The screenshot shows the 'Entregables' page in the S.A.P.S application. The sidebar on the left lists various modules: Dashboard, Usuarios, Cursos, Docentes, Estudiantes, Grupos, Proyectos, Entregables (selected), Observaciones, Rubricas, Criterios, Calificaciones, and Reportes. The main content area features a search bar and a table with 11 deliverables.

ID	Nombre	Proyecto	Tipo	Limite	Estado	Acciones
1	Entregable 1	Plataforma academica integradora 1	PROPUESTA_INICIAL	2026-07-06	PENDIENTE	Editar Eliminar
2	Entregable 2	Plataforma academica integradora 2	AVANCE_1	2026-07-07	PENDIENTE	Editar Eliminar
3	Entregable 3	Plataforma academica integradora 3	AVANCE_2	2026-07-08	PENDIENTE	Editar Eliminar
4	Entregable 4	Plataforma academica integradora 4	INFORME_FINAL	2026-07-09	ENTREGADO	Editar Eliminar
5	Entregable 5	Plataforma academica integradora 5	SUSTENTACION	2026-07-10	PENDIENTE	Editar Eliminar
6	Entregable 6	Plataforma academica integradora 6	REPOSITORIO_GITHUB	2026-07-11	PENDIENTE	Editar Eliminar
7	Entregable 7	Plataforma academica integradora 1	PROTOTIPO	2026-07-12	PENDIENTE	Editar Eliminar
8	Entregable 8	Plataforma academica integradora 2	PROPUESTA_INICIAL	2026-07-13	ENTREGADO	Editar Eliminar
9	Entregable 9	Plataforma academica integradora 3	AVANCE_1	2026-07-14	PENDIENTE	Editar Eliminar
10	Entregable 10	Plataforma academica integradora 4	AVANCE_2	2026-07-15	PENDIENTE	Editar Eliminar
11	Entregable 11	Plataforma academica integradora 5	INFORME_FINAL	2026-07-16	PENDIENTE	Editar Eliminar

Figura 6. Gestión de rúbricas.

The screenshot shows the 'Rubricas' page in the S.A.P.S application. The sidebar on the left lists various modules: Dashboard, Usuarios, Cursos, Docentes, Estudiantes, Grupos, Proyectos, Entregables, Observaciones, Rubricas (selected), Criterios, Calificaciones, and Reportes. The main content area features a search bar and a table with 3 rubrics.

ID	Nombre	Curso	Puntaje	Estado	Acciones
1	Rubrica integradora 1	Lenguaje de Programacion 2	20.00	ACTIVO	Criterios Editar Eliminar
2	Rubrica integradora 2	Ingenieria de Software 2	20.00	ACTIVO	Criterios Editar Eliminar
3	Rubrica integradora 3	Base de Datos 2	20.00	ACTIVO	Criterios Editar Eliminar

S.A.P.S - Sistema Administrador de Proyectos Son
Desarrollado por Son

Figura 7. Registro de calificaciones.

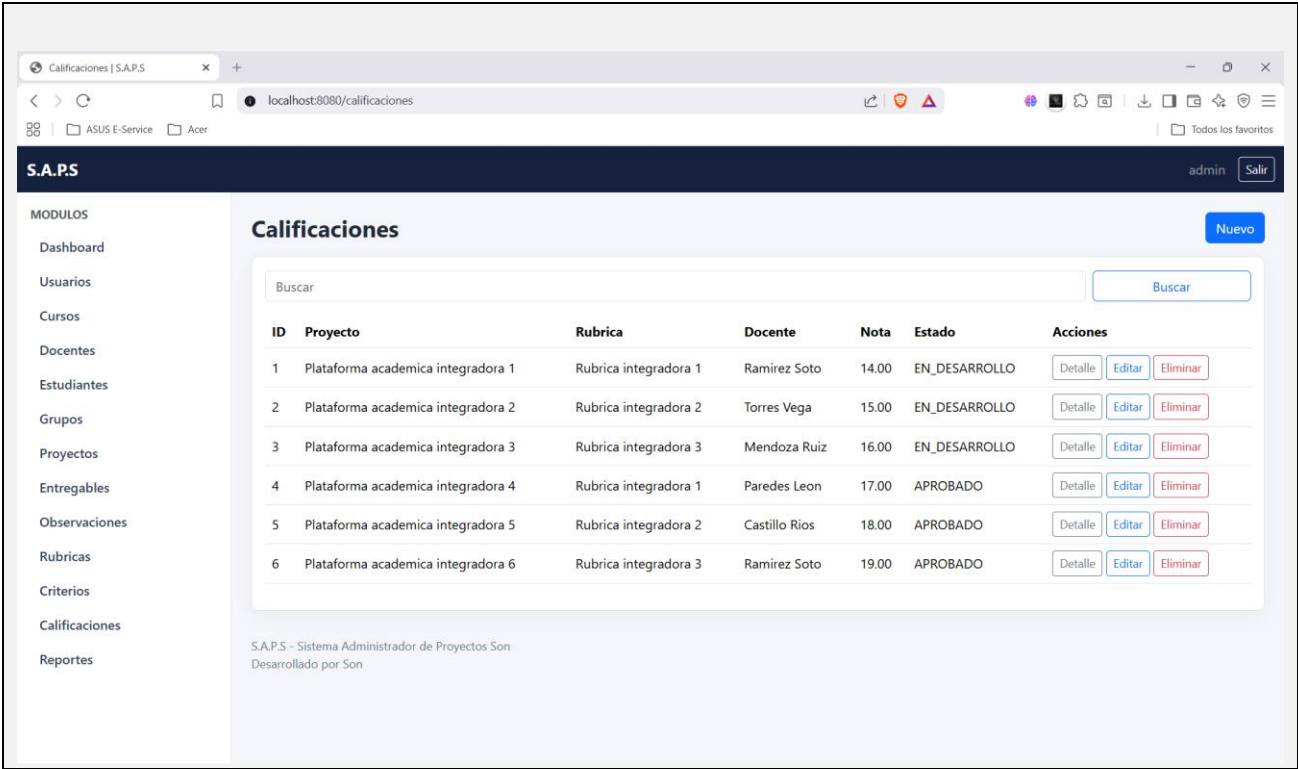
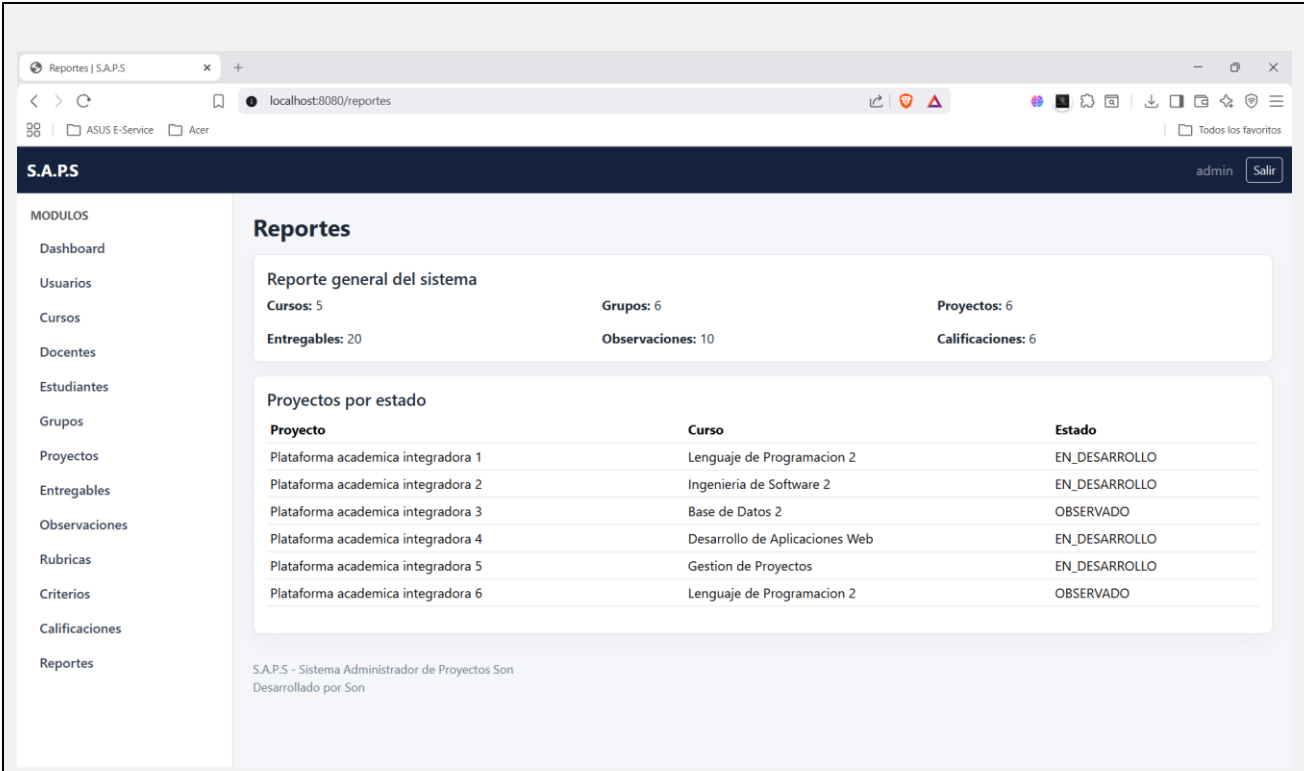


Figura 8. Reportes.



14. Diagramas UML

Los diagramas técnicos se encuentran en la carpeta docs/uml y sirven como apoyo para explicar la estructura del sistema. Los archivos están escritos en PlantUML y Mermaid, lo que permite actualizarlos y renderizarlos con herramientas compatibles.

Tabla 11. Diagramas UML y archivos de soporte.

Diagrama	Archivo	Propósito
Casos de uso	docs/uml/casos_uso.puml	Representar actores y funciones principales del sistema.
Clases	docs/uml/clases.puml	Mostrar entidades relevantes y relaciones del dominio.
Entidad-relación	docs/uml/entidad_relacion.puml	Describir tablas y cardinalidades principales.
Arquitectura MVC	docs/uml/arquitectura_mvc.mmd	Explicar el flujo entre navegador, vistas, controladores, servicios, repositorios y MySQL.
Flujo principal	docs/uml/flujo_principal.mmd	Mostrar el recorrido desde registro de proyecto hasta reporte.

15. Pruebas del sistema

Las pruebas registradas son funcionales y manuales. Se orientan a verificar que la aplicación pueda iniciar, autenticar usuarios, mostrar módulos y registrar información principal. La matriz completa se mantiene también en docs/Pruebas_SAPS.md.

Tabla 12A. Matriz de pruebas funcionales: autenticación y mantenimiento.

Código	Módulo	Caso de prueba	Resultado verificado	Estado
CP-001	Autenticación	Login administrador	Acceso correcto al dashboard con admin/admin123.	Aprobado
CP-002	Autenticación	Login docente	Credencial disponible y acceso correcto con docente/docente123.	Aprobado
CP-003	Autenticación	Login estudiante	Credencial disponible y acceso correcto con estudiante/estudiante123.	Aprobado
CP-004	Cursos	Registro de curso	Formulario disponible y curso guardado con datos válidos.	Aprobado
CP-005	Estudiantes	Registro de estudiante	Formulario disponible y estudiante guardado con datos válidos.	Aprobado
CP-006	Grupos	Creación de grupo	Grupo guardado y vista de detalle disponible.	Aprobado
CP-007	Proyectos	Registro de proyecto	Proyecto guardado; listado y detalle disponibles.	Aprobado
CP-008	Entregables	Registro de entregable	Entregable guardado mediante formulario.	Aprobado

Tabla 12B. Matriz de pruebas funcionales: evaluación, dashboard y reportes.

Código	Módulo	Caso de prueba	Resultado verificado	Estado
CP-009	Observaciones	Registro de observación	Observación guardada con proyecto, docente y comentario.	Aprobado
CP-010	Rúbricas	Creación de rúbrica	Rúbrica guardada y listado disponible.	Aprobado
CP-011	Criterios	Registro de criterio	Criterio guardado con rúbrica, puntaje y peso.	Aprobado
CP-012	Calificaciones	Registro de calificación	Calificación guardada; listado y detalle disponibles.	Aprobado
CP-013	Calificaciones	Cálculo de nota final	Servicio implementa cálculo ponderado cuando existen detalles.	Aprobado técnico

Código	Módulo	Caso de prueba	Resultado verificado	Estado
CP-014	Dashboard	Visualización del dashboard	Indicadores y resúmenes cargan correctamente.	Aprobado
CP-015	Reportes	Consulta de reportes	Reporte general disponible desde /reportes.	Aprobado

16. Resultados obtenidos

Como resultado obtuve una aplicación web académica funcional que integra módulos de mantenimiento, seguimiento y evaluación. El sistema permite trabajar con datos iniciales, autenticar usuarios, gestionar entidades principales y consultar información desde dashboard y reportes.

- Se implementó una estructura MVC clara con controladores, servicios, repositorios, entidades y vistas.
- Se integró Thymeleaf con formularios, listados, expresiones th:* y fragmentos reutilizables.
- Se usó Spring Data JPA para persistencia y consultas derivadas.
- Se configuró MySQL como base relacional local y Hibernate para creación o actualización de tablas.
- Se implementó seguridad con login personalizado, roles y restricción de usuarios al rol administrador.
- Se documentaron pruebas funcionales y diagramas UML de apoyo.

17. Alineación con la rúbrica de evaluación

Para evidenciar la relación directa entre el sistema y los criterios de evaluación del curso, se presenta la siguiente matriz de alineación. Esta sección sirve también como guía para la sustentación oral, porque resume qué evidencia debe mostrarse durante la demostración.

Tabla 13. Alineación del proyecto con la rúbrica.

Criterio de rúbrica	Evidencia en S.A.P.S	Cómo sustentarlo
Implementación de MVC con Spring Boot	Controladores, entidades, servicios, repositorios y vistas Thymeleaf organizados por paquetes.	Explicar el flujo Usuario → Vista → Controlador → Servicio → Repositorio → Base de datos.
Integración y uso de Thymeleaf	Vistas con formularios, listados, fragmentos, expresiones th:* y componentes Bootstrap.	Mostrar login, dashboard, listado de proyectos y formulario de registro.
Manejo de base de datos con Spring Data JPA	Entidades JPA, relaciones, repositorios JpaRepository, MySQL y scripts de base de datos.	Mostrar el modelo de entidades y explicar relaciones como Curso-Proyecto, Proyecto-Entregable y Rúbrica-Criterio.
Documentación y justificación técnica	Informe, README, manuales, diagramas UML, matriz de pruebas y explicación de tecnologías.	Usar el informe para sustentar problema, objetivos, arquitectura, módulos, pruebas y conclusiones.
Exposición y dominio del tema	Guion de sustentación, flujo de demostración y explicación de decisiones técnicas.	Presentar primero el problema, luego arquitectura, después módulos y finalmente resultados.

18. Limitaciones y mejoras futuras

Las limitaciones identificadas no invalidan el proyecto; representan oportunidades de evolución para una versión posterior.

- Implementar exportación de reportes a PDF o Excel.
- Agregar carga real de archivos para entregables, ya que actualmente se registra una URL.

- Definir reglas de autorización más específicas por módulo para DOCENTE y ESTUDIANTE.
- Crear una interfaz completa para registrar detalle de calificación por criterio desde la pantalla de calificaciones.
- Incorporar notificaciones por correo para observaciones o entregables próximos.
- Añadir historial de cambios y auditoría de operaciones relevantes.
- Incluir gráficos estadísticos más avanzados para el dashboard.

19. Conclusiones

1. El proyecto cumple el objetivo general al implementar una aplicación web MVC para administrar proyectos integradores universitarios con Spring Boot, Thymeleaf, JPA y MySQL.
2. La arquitectura por capas permitió separar responsabilidades y facilitó la explicación técnica del sistema durante la sustentación.
3. El uso de Thymeleaf permitió construir vistas dinámicas integradas con los modelos enviados por los controladores.
4. Spring Data JPA simplificó la persistencia mediante repositorios y relaciones entre entidades del dominio académico.
5. El desarrollo del sistema fortaleció mi aprendizaje en Java, Spring Boot, seguridad, validaciones, formularios web, base de datos y documentación técnica.

20. Recomendaciones

1. Antes de ejecutar el sistema, activar MySQL desde XAMPP y verificar que la base saps_db exista.
2. Ejecutar el proyecto con Java 17 para mantener compatibilidad con la configuración Maven y NetBeans.
3. Crear la base de datos con `CREATE DATABASE saps_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci`; antes de la primera ejecución.
4. Revisar `application.properties` si se cambia usuario, contraseña, puerto o motor de base de datos.
5. Mantener la estructura por capas para futuras funcionalidades y evitar mezclar lógica de negocio en las vistas.
6. Durante la sustentación, iniciar por el problema, continuar con arquitectura y cerrar con una demostración de módulos y reportes.

21. Referencias

- Spring Boot. Documentación oficial del proyecto. <https://spring.io/projects/spring-boot/>
- Spring Data JPA. Documentación de referencia. <https://docs.spring.io/spring-data/jpa/reference/>
- Thymeleaf. Documentación oficial. <https://www.thymeleaf.org/documentation.html>
- MySQL. Documentación oficial. <https://dev.mysql.com/doc/>
- Bootstrap 5.3. Documentación oficial. <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- Repositorio interno del proyecto S.A.P.S: archivos README.md, pom.xml, application.properties, src/main/java, src/main/resources/templates, database y docs/uml.